

Universidad Nacional de Colombia

Trabajo 1

Departamento de de Ciencias de la Computación y de la Decisión
Facultad de Minas

Integrantes:

Juan Manuel Diaz Larrea
Kevin Hair González Duarte
Manuel Gutierrez Tangarife
Mariana Ruiz Zamora
Maria Fernanda Valencia Jimenez

Medellín, Colombia
Semestre 2026 - 02
13 de octubre de 2026

Índice

1. Repositorio GitHub **5**

2. Video **5**

3. Parte 1 **5**
3.1. Fundamentación matemática: funciones de prueba . . . 5
3.1.1. Función de Rosenbrock . . . 5
3.1.2. Función de Rastrigin . . . 5
3.2. Métodos y configuración . . . 6
3.2.1. Sintonización de hiperparámetros . . . 6
3.3. Regla de comparación justa . . . 6
3.3.1. Protocolo estadístico . . . 7
3.4. Resultados . . . 7
3.5. Discusión . . . 10
3.5.1. Qué aportó el descenso por gradiente . . . 10
3.5.2. Qué aportaron los métodos heurísticos . . . 11
3.5.3. Efecto de la dimensión y del umbral . . . 11
3.5.4. Síntesis . . . 11
3.6. Limitaciones . . . 12
3.7. Conclusiones . . . 12

4. Parte 2 **12**
4.1. Contexto geográfico: España peninsular . . . 12
4.2. Marco teórico . . . 13
4.3. Fundamentación matemática: . . . 14
4.3.1. Colonia de hormigas (ACO) . . . 14
4.3.2. Algoritmo genético (GA) . . . 14
4.3.3. Búsqueda local (Para luego hacer comparación) . . . 15
4.3.4. Verificación . . . 15

5. Provincias españolas y sus capitales **16**

6. Datos geográficos y matrices de tiempo y distancia **16**
6.1. Origen de las coordenadas . . . 16
6.2. Por qué se necesitan tiempos y distancias por carretera . . . 17
6.3. Primer intento: API pública de OpenRouteService . . . 17
6.4. Error detectado: rutas por carreteras de Portugal . . . 17
6.5. Solución: servidor local de OpenRouteService . . . 18
6.6. Por qué fue importante el cambio . . . 18
6.7. Consecuencias de no haber hecho el cambio . . . 18
6.8. Supuestos y limitaciones . . . 18

7. Solución punto 1. **19**
7.1. Mejor recorrido encontrado . . . 19
7.2. Comparación con búsqueda local . . . 20

8. Solución punto 2. **20**

9. Parámetros económicos y función de costo **20**
9.1. Vehículo de referencia . . . 21
9.2. Precio del combustible . . . 21
9.3. Conversión de la distancia en costo de combustible . . . 22
9.4. Peajes . . . 22
9.5. Valor de la hora . . . 23
9.6. Función de costo completa . . . 24
9.7. Sensibilidad al valor de la hora . . . 24
9.8. Limitaciones . . . 24
9.9. Resultados . . . 25

10. USO DE IA **26**
10.1. Punto 1 . . . 26
10.2. Punto 2 . . . 28
10.3. Fase 1: Comprensión del Problema y Contexto Teórico . . . 28
10.4. Fase 2: Obtención de Datos y Detección de Errores . . . 28
10.5. Fase 3: Implementación del Servidor Local (Docker) . . . 28

11. Resultados, discusión y conclusiones **29**
11.1. Resultados del proceso . . . 29
11.2. Discusión . . . 29
11.3. Conclusiones . . . 29

Índice de figuras

1. Distribución de $\log_{10}(f \text{ final})$ en 30 corridas. El valor -30 corresponde a $f = 0$ exacto (se trunca para poder graficar). . . . 9
2. Boxplot de los valores de f para los diferentes algoritmos y dimensiones. . . . 10
3. Mapa físico de la Península Ibérica. Fuente: Emilio Gómez Fernández, Wikimedia Commons, licencia CC BY-SA 3.0. . . . 13
4. Mejores recorridos hallados por ACO (izquierda) y GA (centro); las flechas indican el sentido. A la derecha, convergencia de la mejor longitud encontrada frente al óptimo exacto (línea discontinua). . . . 19
5. Mejor recorrido encontrado en cada escenario, obtenido en ambos casos con ACO (semilla 20261008). . . . 25

Índice de cuadros

1. Hiperparámetros elegidos por la búsqueda en rejilla. . . . 6
2. Rosenbrock 2D (30 corridas, 10 000 evaluaciones equivalentes). Éxito: $f < 10^{-4}$. . . 7
3. Rosenbrock 3D. Éxito: $f < 10^{-4}$. . . . 7
4. Rastrigin 2D. Éxito: $f < 10^{-2}$. . . . 8
5. Rastrigin 3D. Éxito: $f < 10^{-2}$. . . . 8
6. Evaluaciones equivalentes promedio hasta alcanzar el umbral; entre paréntesis, las corridas exitosas (de 30). "—" indica que ninguna corrida alcanzó el umbral. . 8
7. **De Álava a Córdoba** . . . 16
8. **De A Coruña a Murcia** . . . 16
9. **De Navarra a Zaragoza** . . . 16
10. Provincias y capitales de la España Peninsular . . . 16
11. Resumen de resultados (distancias por carretera, semilla 42). . . . 19
12. Mejor recorrido encontrado (GA), tramo a tramo (km por carretera). . . . 20

13. Características del vehículo de referencia 21
14. Mejor recorrido encontrado por cada algoritmo y escenario (semilla 20261008). . 25

1. Repositorio GitHub

https://github.com/MgtDevNet/T01_REDES_NEURONALES_

2. Video

https://youtu.be/By5gqy9wKyw

3. Parte 1

3.1. Fundamentación matemática: funciones de prueba

Antes de comparar algoritmos se necesitan problemas de referencia con propiedades matemáticas conocidas: dominio, ubicación del mínimo global y complejidad de la superficie. Se seleccionaron las funciones de Rosenbrock y Rastrigin (Rosenbrock 1960; Rastrigin 1974), que son contrastantes: la primera presenta un valle estrecho y curvo que dificulta la convergencia hacia el mínimo global, mientras que la segunda es altamente multimodal y exige exploración global. Así permiten evaluar tanto la explotación de los métodos basados en gradiente como la exploración de los metaheurísticos.

3.1.1. Función de Rosenbrock

También conocida como la *función banana*, se define para $\mathbf{x} \in \mathbb{R}^d$ como

$$f(\mathbf{x}) = \sum_{i=1}^{d-1} \left[100\,(x_{i+1} - x_i^2)^2 + (1 - x_i)^2\right]. \tag{1}$$

Dominio. En los experimentos se restringe a $\mathbf{x} \in [-2{,}048,\ 2{,}048]^d$.
Mínimo global. Se alcanza en $\mathbf{x}^* = (1, \ldots, 1)$, con $f(\mathbf{x}^*) = 0$.
Geometría. No es convexa, pero en 2D y 3D tiene un único mínimo global (para $d \geq 4$ aparece además un mínimo local cerca de $(-1, 1, \ldots, 1)$). Su dificultad es el condicionamiento: el óptimo está en un valle parabólico estrecho y curvo ($x_2 \approx x_1^2$). En 2D, el Hessiano en el óptimo es

$$H = \begin{pmatrix} 802 & -400 \\ -400 & 200 \end{pmatrix},$$

con autovalores $\approx 1001{,}6$ y $\approx 0{,}4$, es decir, un número de condición $\kappa \approx 2500$. Un método de gradiente debe usar pasos pequeños por la dirección de mayor curvatura y avanza muy lentamente a lo largo del valle.

3.1.2. Función de Rastrigin

Es una función multimodal con mínimos locales distribuidos periódicamente, definida como

$$f(\mathbf{x}) = 10d + \sum_{i=1}^{d} \left[x_i^2 - 10\cos(2\pi x_i)\right]. \tag{2}$$

Dominio. Se considera $\mathbf{x} \in [-5{,}12,\ 5{,}12]^d$.
Mínimo global. Se encuentra en $\mathbf{x}^* = (0, \ldots, 0)$, con $f(\mathbf{x}^*) = 0$.
Geometría. Tiene un mínimo local cerca de cada punto de coordenadas enteras, es decir, del orden de 11^d mínimos en el dominio (≈ 121 en 2D y $\approx 1\,331$ en 3D). Cada pozo es localmente convexo y bien condicionado (la segunda derivada por coordenada en el origen es $2 + 40\pi^2 \approx 397$), de modo que la dificultad no es el condicionamiento sino la exploración global: un método local converge al pozo donde empieza.

3.2. Métodos y configuración

Todos los métodos comparten la misma interfaz y el mismo presupuesto máximo de evaluaciones equivalentes. Los dominios son los de cada función y las soluciones se restringen a ellos.

- **GD proyectado con Armijo.** Gradiente por diferencias centrales ($h = 10^{-6}$), paso inicial α_0 con retroceso ($\alpha \leftarrow \text{factor} \cdot \alpha$), condición de Armijo proyectada con $c = 10^{-4}$ y proyección al dominio. El único elemento aleatorio es el punto inicial uniforme. Se detiene si $\|\nabla f\| < 10^{-8}$ o si la búsqueda lineal falla.
- **GA real.** Selección por torneo ($k = 3$), cruce BLX-α, mutación gaussiana por gen con probabilidad $1/d$ y desviación decreciente de forma geométrica (de 0,1 a 10^{-4} del rango), y elitismo de 2 individuos.
- **PSO.** Topología global con constricción de Clerc–Kennedy ($c_1 = c_2 = 1{,}49618$), velocidad máxima del 20 % del rango y velocidad nula al tocar la frontera.
- **DE/rand/1/bin.** Mutante $\mathbf{v} = \mathbf{x}_a + F(\mathbf{x}_b - \mathbf{x}_c)$, cruce binomial con al menos un gen del mutante y selección voraz uno a uno.
- **Líneas base.** Búsqueda aleatoria uniforme y recocido simulado (Metropolis, enfriamiento geométrico, vecino gaussiano).

3.2.1. Sintonización de hiperparámetros

Se hizo una búsqueda en rejilla sobre Rosenbrock 2D con 4 000 evaluaciones y las semillas 10, 20 y 30, eligiendo la combinación con menor mediana de f. Las configuraciones elegidas se usaron en todos los escenarios (Tabla 1).

Método	Hiperparámetros elegidos
GD	`tasa_aprendizaje_inicial` $= 0{,}5$; `factor_reduccion` $= 0{,}3$
GA	`tam_poblacion` $= 30$; `prob_cruce` $= 0{,}95$; `alpha_blx` $= 0{,}3$
PSO	`inercia` $= 0{,}5$; `num_particulas` $= 30$
DE	$F = 0{,}5$; $CR = 0{,}9$; `tam_poblacion` $= 30$
Búsqueda aleatoria y recocido	Valores por defecto (sin sintonizar)

Cuadro 1: Hiperparámetros elegidos por la búsqueda en rejilla.

3.3. Regla de comparación justa

Comparar por número de iteraciones favorece a quien hace más trabajo por iteración, así que se contabiliza el costo real en evaluaciones de la función. Se reportan por separado el número de evaluaciones directas de f (n_f: punto inicial, poblaciones y pruebas de la búsqueda lineal) y el número de gradientes calculados (n_{grad}).

Equivalencia propuesta. Un gradiente por diferencias centrales requiere $f(\mathbf{x} + h\mathbf{e}_i)$ y $f(\mathbf{x} - h\mathbf{e}_i)$ para cada una de las d direcciones canónicas, es decir, exactamente $2d$ evaluaciones de f. Por tanto:

$$n_{\text{equiv}} = n_f + 2d\, n_{\text{grad}}. \tag{3}$$

Las $2d$ evaluaciones internas del gradiente no se suman a n_f: se cuentan una sola vez, dentro de n_{grad}. Para los métodos sin derivadas, $n_{\text{grad}} = 0$ y $n_{\text{equiv}} = n_f$. El código verifica el conteo con una aserción: las llamadas reales a f deben ser iguales a $n_f + 2d\, n_{\text{grad}}$. El presupuesto es el mismo para todos los métodos: 10 000 evaluaciones equivalentes.

Esta equivalencia supone un gradiente numérico por diferencias centrales. Con diferencias hacia adelante el costo sería $d + 1$ evaluaciones y con un gradiente analítico o diferenciación automática sería un pequeño múltiplo del costo de f, lo que favorecería más a GD; se discute en la Sección 3.6.

3.3.1. Protocolo estadístico

- 30 corridas independientes por configuración (semillas 0 a 29), para cada método, función y dimensión.
- Estadísticos del valor final de f: media, desviación estándar (muestral, ddof $= 1$), mediana, mejor y peor valor.
- Tasa de éxito: fracción de corridas con f final por debajo de un umbral. Umbrales definidos: 10^{-4} para Rosenbrock (cercanía al óptimo global en el valle) y 10^{-2} para Rastrigin (distingue el pozo global de cualquier mínimo local, cuyo valor mínimo es $\approx 0{,}995$).
- Además se reporta el promedio de evaluaciones equivalentes necesarias para alcanzar el umbral, calculado solo sobre las corridas exitosas.

3.4. Resultados

Las Tablas 2–5 resumen el valor final de f y el costo en evaluaciones (n_f, n_{grad} y n_{equiv}) para cada escenario; la Tabla 6 muestra cuántas evaluaciones necesita cada método para alcanzar el umbral.

Método	Media	Desv. est.	Mejor	Peor	Éxito	n_f	n_{grad}	n_{equiv}
GD	0.0122	0.0284	3.1-7	0.112	23.3 %	5734	1066	9999
GA	9.0-10	1.5-9	0	7.7-9	100.0 %	9998	0	9998
PSO	3.6-17	1.5-16	2.4-27	8.4-16	100.0 %	9990	0	9990
DE	0	0	0	0	100.0 %	9990	0	9990
Búsqueda aleatoria	6.6-3	6.7-3	2.5-6	0.0302	3.3 %	10000	0	10000
Recocido simulado	2.3-4	2.8-4	1.8-6	1.3-3	36.7 %	10000	0	10000

Cuadro 2: Rosenbrock 2D (30 corridas, 10 000 evaluaciones equivalentes). Éxito: $f < 10^{-4}$.

Método	Media	Desv. est.	Mejor	Peor	Éxito	n_f	n_{grad}	n_{equiv}
GD	0.141	0.723	8.6-5	3.97	3.3 %	4809	865	9998
GA	0.175	0.536	3.9-3	3	0.0 %	9998	0	9998
PSO	2.0-3	1.3-3	2.2-4	4.5-3	0.0 %	9990	0	9990
DE	0.0224	0.0515	0	0.277	36.7 %	9990	0	9990
Búsqueda aleatoria	0.43	0.299	0.062	1.01	0.0 %	10000	0	10000
Recocido simulado	0.0119	0.0114	5.2-4	0.0497	0.0 %	10000	0	10000

Cuadro 3: Rosenbrock 3D. Éxito: $f < 10^{-4}$.

Método	Media	Desv. est.	Mejor	Peor	Éxito	n_f	n_{grad}	n_{equiv}
GD	5.6	4.83	0	19.9	13.3 %	2515	320	3794
GA	3.2-9	5.3-9	0	2.2-8	100.0 %	9998	0	9998
PSO	0.0995	0.304	0	0.995	90.0 %	9990	0	9990
DE	0	0	0	0	100.0 %	9990	0	9990
Búsqueda aleatoria	0.609	0.367	0.0473	1.19	0.0 %	10000	0	10000
Recocido simulado	4.32	4.84	1.6-4	17.9	3.3 %	10000	0	10000

Cuadro 4: Rastrigin 2D. Éxito: $f < 10^{-2}$.

Método	Media	Desv. est.	Mejor	Peor	Éxito	n_f	n_{grad}	n_{equiv}
GD	16.3	10.7	0.995	40.8	0.0 %	3738	384	6041
GA	0.0995	0.304	1.0-10	0.995	90.0 %	9998	0	9998
PSO	0.398	0.671	0	2.98	66.7 %	9990	0	9990
DE	0	0	0	0	100.0 %	9990	0	9990
Búsqueda aleatoria	3.12	1.1	0.931	5.11	0.0 %	10000	0	10000
Recocido simulado	9.11	6.07	1.02	22.6	0.0 %	10000	0	10000

Cuadro 5: Rastrigin 3D. Éxito: $f < 10^{-2}$.

Método	Rosenbrock 2D	Rosenbrock 3D	Rastrigin 2D	Rastrigin 3D
GD	7308 (7/30)	8944 (1/30)	74 (4/30)	—
GA	2157 (30/30)	—	999 (30/30)	1561 (27/30)
PSO	1450 (30/30)	—	744 (27/30)	1521 (20/30)
DE	867 (30/30)	2032 (11/30)	1295 (30/30)	2976 (30/30)
Búsqueda aleatoria	8761 (1/30)	—	—	—
Recocido simulado	7024 (11/30)	—	2025 (1/30)	—

Cuadro 6: Evaluaciones equivalentes promedio hasta alcanzar el umbral; entre paréntesis, las corridas exitosas (de 30). "—" indica que ninguna corrida alcanzó el umbral.

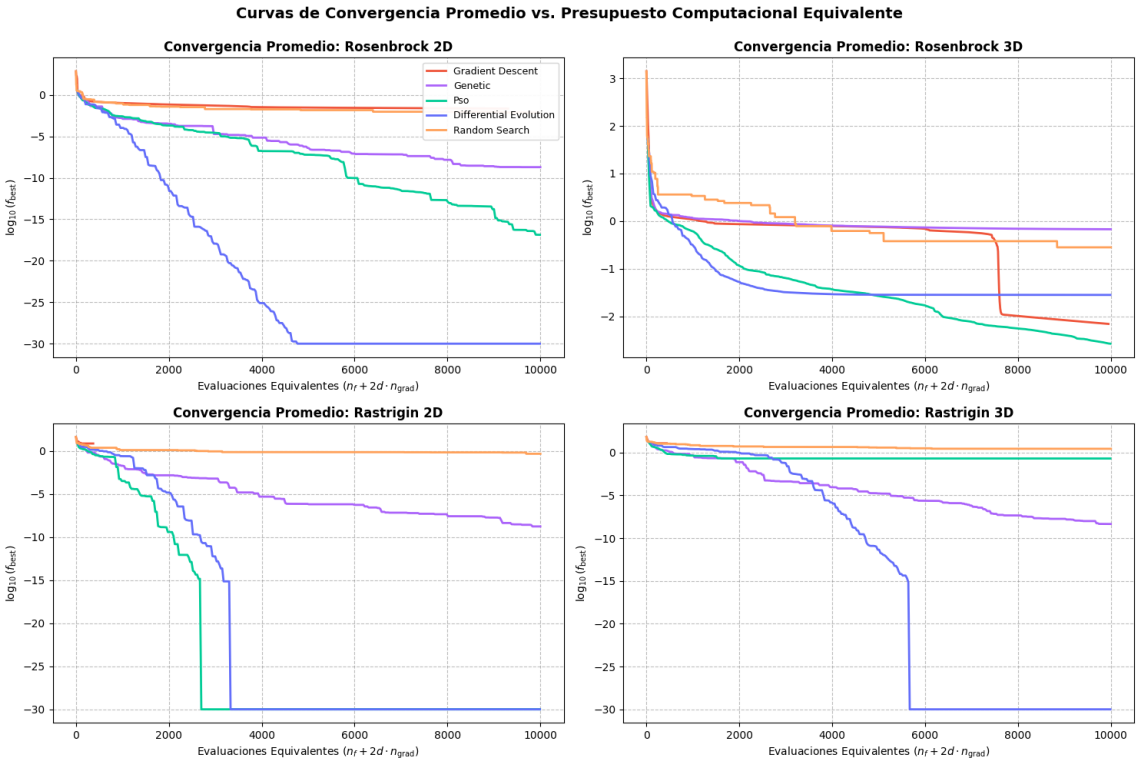


Figura 1: Distribución de $\log_{10}(f \text{ final})$ en 30 corridas. El valor -30 corresponde a $f = 0$ exacto (se trunca para poder graficar).

En la Figura 1 se ilustra la velocidad y trayectoria de convergencia promedio de cada algoritmo a lo largo del presupuesto de evaluaciones computacionales equivalentes ($n_f + 2d \cdot n_{\text{grad}}$). A partir de las curvas se evidencia de forma contundente que **Evolución Diferencial (Differential Evolution - DE)** es el método con el mejor desempeño general de todo el estudio: no solo exhibe la pendiente de descenso más pronunciada en las etapas tempranas, sino que es el único algoritmo capaz de alcanzar el límite numérico de precisión ($\log_{10}(f_{\text{best}}) \approx -30$) de manera sistemática tanto en espacios continuos unimodales con crestas estrechas (Rosenbrock 2D) como en paisajes altamente multimodales y rugosos (Rastrigin 2D y 3D), requiriendo menos de 6 000 evaluaciones equivalentes para colapsar en el mínimo global. Si bien PSO muestra una rápida caída inicial en Rastrigin 2D y un progreso competitivo en Rosenbrock 3D, sufre estancamiento prematuro en Rastrigin 3D; mientras que el Algoritmo Genético converge con lentitud y tanto Descenso por Gradiente como la Búsqueda Aleatoria presentan mesetas tempranas sin capacidad de escape. Por consiguiente, el operador de mutación vectorial guiado de Evolución Diferencial ofrece el mejor equilibrio entre exploración global y explotación local frente a cualquier otra técnica evaluada.

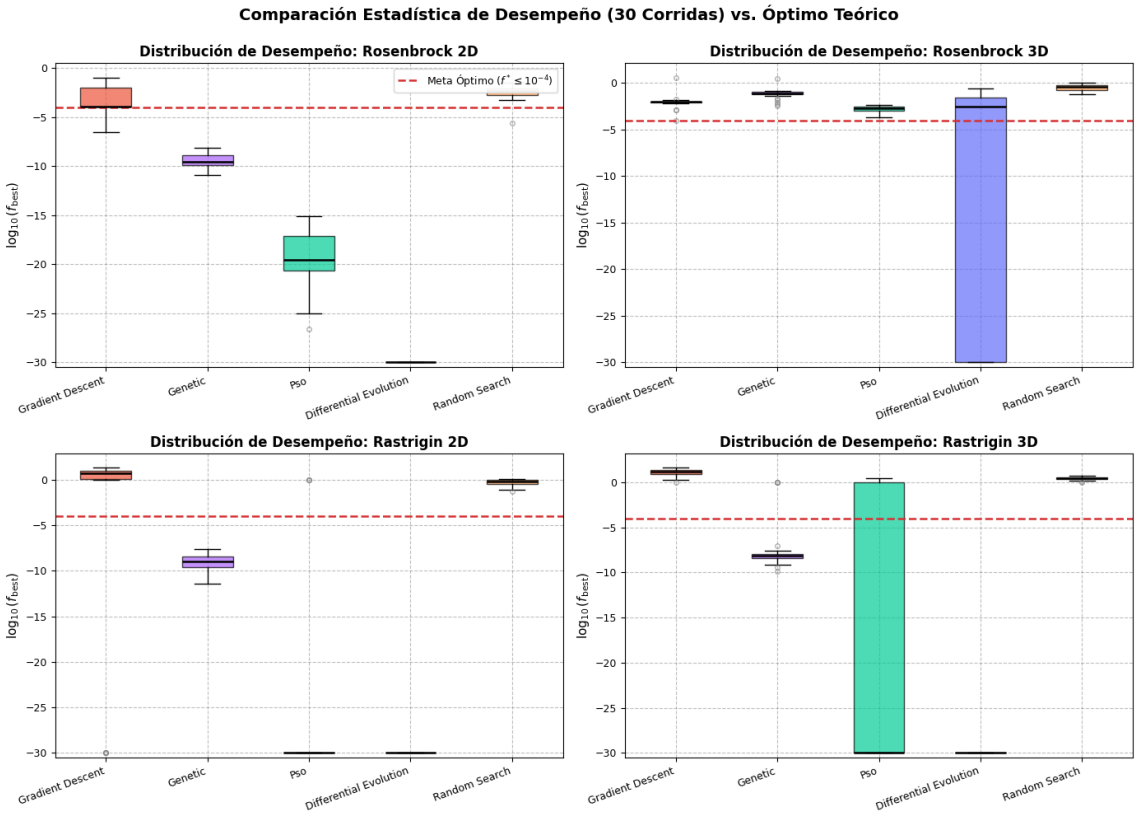


Figura 2: Boxplot de los valores de f para los diferentes algoritmos y dimensiones.

En la Figura 2 se presentan los boxplots de los valores finales de f en 30 corridas. La línea roja discontinua marca el umbral de éxito usado para Rosenbrock, $f < 10^{-4}$; para Rastrigin se adopta el umbral distinto $f < 10^{-2}$. DE muestra la mayor tasa de éxito global: 100 % en Rosenbrock 2D y Rastrigin 2D/3D, y 36.7 % en Rosenbrock 3D. GA alcanza 100 % en Rastrigin 2D y 90 % en 3D, pero no logra el umbral en Rosenbrock 3D. PSO alcanza 100 % en Rosenbrock 2D y 90 %/66.7 % en Rastrigin 2D/3D, pero no alcanza el umbral en Rosenbrock 3D. GD obtiene tasas de éxito de 23.3 % y 3.3 % en Rosenbrock 2D/3D, y de 13.3 % y 0 % en Rastrigin 2D/3D. En conjunto, la figura muestra que el desempeño depende del algoritmo, la función y la dimensión.

3.5. Discusión

3.5.1. Qué aportó el descenso por gradiente

- **Eficiencia local y convexidad local.** Cuando el problema es localmente convexo y bien condicionado, GD usa pocas evaluaciones. En Rastrigin cada pozo cumple esa condición: GD consume en promedio 3 794 (2D) y 6 041 (3D) de las 10 000 evaluaciones equivalentes, converge rápido al mínimo local del pozo donde empieza, el gradiente se anula y se detiene sin agotar el presupuesto.

- **Límite por condicionamiento (Rosenbrock).** Con $\kappa \approx 2500$ en el óptimo, la búsqueda de Armijo reduce el paso y el avance a lo largo del valle es lento. GD agota el presupuesto ($\approx$ 1 066 gradientes en 2D, 865 en 3D) y solo alcanza el umbral en 23,3 % de las corridas en 2D y en 3,3 % en 3D; la media (1.2-2 y 0,14) está dominada por unas pocas corridas malas.

- **Límite por multimodalidad (Rastrigin).** Con $\approx$ 121 mínimos en 2D y $\approx$ 1 331 en 3D, GD

termina en pozos locales: f media 5,6 en 2D y 16,3 en 3D, y éxito de 13,3 % y 0 %. El 13,3 % en 2D es mayor que la probabilidad de empezar dentro del pozo global ($\approx$ 1 %); una explicación plausible es que con paso inicial 0,5 y gradientes de decenas de unidades las primeras iteraciones saltan entre pozos antes de que el paso se reduzca. Es una hipótesis que no se verificó aquí.

- **Costo creciente con d.** Cada gradiente cuesta $2d$ evaluaciones (4 en 2D, 6 en 3D), así que pasar a 3D reduce los pasos disponibles con el mismo presupuesto.

3.5.2. Qué aportaron los métodos heurísticos

- **DE** fue el método más robusto: 100 % de éxito en Rosenbrock 2D y en Rastrigin 2D y 3D, con $f = 0$ exacto, y 36,7 % en Rosenbrock 3D (el mejor de ese escenario). Las diferencias entre individuos se adaptan a la forma de la población: se alargan a lo largo del valle de Rosenbrock y permiten explorar varios pozos de Rastrigin a la vez.
- **PSO** es muy preciso en Rosenbrock 2D (media 3.6-17) y tiene la menor dispersión en Rosenbrock 3D (mediana $\approx$ 1.8-3, con 0 % de éxito solo por lo estricto del umbral 10^{-4}). En Rastrigin pierde fiabilidad (90 % en 2D, 66,7 % en 3D): el enjambre converge hacia el mejor global y a veces lo hace en un pozo local.
- **GA** es muy bueno en Rastrigin (100 % en 2D, 90 % en 3D), donde la mutación gaussiana decreciente refina bien dentro del pozo global, pero es lento en el valle de Rosenbrock 3D (0 % de éxito, media 0,175).
- **Costo.** Los heurísticos no usan gradiente, de modo que consumen casi todo el presupuesto. Las líneas base muestran que su ventaja no es solo cantidad de evaluaciones: con el mismo presupuesto, la búsqueda aleatoria casi nunca alcanza el umbral (0 % salvo 3,3 % en Rosenbrock 2D) y el recocido simulado sin sintonizar es débil excepto en Rosenbrock 2D (36,7 %).

3.5.3. Efecto de la dimensión y del umbral

Pasar de 2D a 3D empeora a todos los métodos: crece el volumen del espacio de búsqueda, crece el número de pozos de Rastrigin ($\approx$ 121 a $\approx$ 1 331) y el valle de Rosenbrock se complica. En Rosenbrock 3D ningún método es fiable con el umbral 10^{-4} en 10 000 evaluaciones; la conclusión sobre PSO depende del umbral, porque su mediana está cerca de 10^{-3} con muy poca dispersión. Por eso se recomienda interpretar siempre el éxito junto con la distribución del valor final (Figura 1) y las evaluaciones necesarias (Tabla 6).

3.5.4. Síntesis

El descenso por gradiente aporta eficiencia local: pocas evaluaciones y buen valor final si la cuenca inicial es la correcta, pero no explora y se degrada con el mal condicionamiento. Los heurísticos aportan exploración global y tolerancia a muchos mínimos locales a cambio de gastar el presupuesto completo. La geometría explica el reparto: Rosenbrock (un solo mínimo, valle mal condicionado) premia a quienes se adaptan a la forma del valle (DE, PSO), y Rastrigin (muchísimos mínimos localmente convexos y bien condicionados) premia a quienes exploran (DE, GA). Una estrategia híbrida —heurístico para ubicar la cuenca y GD para refinar— es una continuación natural, pero no se evaluó en este estudio.

3.6. Limitaciones

- Los hiperparámetros se sintonizaron solo en Rosenbrock 2D con 3 semillas y 4 000 evaluaciones, y se reutilizaron en los demás escenarios; por ejemplo, la inercia 0,5 elegida para PSO favorece explotar y puede perjudicarlo en Rastrigin. Además, las semillas 10 y 20 de la sintonización también están entre las 30 semillas de evaluación.
- La equivalencia de $2d$ evaluaciones por gradiente es una elección. Con gradiente analítico, GD haría mucho más trabajo útil con el mismo presupuesto y su desempeño en Rosenbrock mejoraría; conviene repetir el análisis con costos de gradiente distintos (por ejemplo 1, $d+1$ y $2d$).
- Un único presupuesto (10 000), dos funciones y dimensiones 2 y 3; no se aplicaron pruebas de hipótesis (por ejemplo Wilcoxon o Mann–Whitney) para comparar pares de métodos.
- Las líneas base no se sintonizaron.

3.7. Conclusiones

- Con el mismo presupuesto en evaluaciones equivalentes, DE fue el método más confiable en las cuatro configuraciones; PSO y GA fueron competitivos según la función.
- GD es el más barato cuando la cuenca inicial es buena, pero falla en Rastrigin (0 % en 3D) y es lento en Rosenbrock por el condicionamiento.
- Conviene reportar siempre n_f y n_{grad} por separado, la equivalencia usada y varias corridas con distribución completa, porque la media sola oculta fallos (por ejemplo, GD en Rosenbrock 3D).
- Rosenbrock 3D con umbral 10^{-4} sigue siendo difícil para todos los métodos en 10 000 evaluaciones.

4. Parte 2

4.1. Contexto geográfico: España peninsular

Antes de abordar el problema del vendedor viajero, es necesario establecer con claridad el ámbito geográfico en el que se desarrolla el problema. El término *España peninsular* hace referencia a la parte del territorio español situada en la Península Ibérica. Por tanto, no constituye un país diferente de España, sino una delimitación geográfica del territorio español.

La Península Ibérica se encuentra en el extremo suroccidental de Europa y está ocupada principalmente por España y Portugal, además de incluir los territorios de Andorra y Gibraltar. España, por su parte, posee territorios que no se encuentran en la península, entre ellos las Islas Baleares, las Islas Canarias y las ciudades autónomas de Ceuta y Melilla. En consecuencia, cuando se utiliza el término *España peninsular*, se excluyen estos territorios y se considera únicamente la parte de España ubicada en la Península Ibérica.

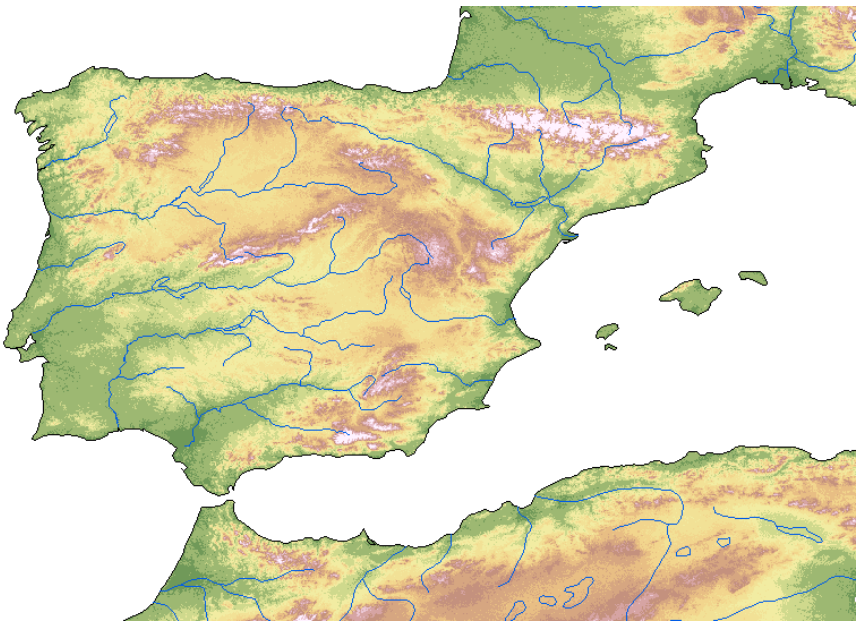


Figura 3: Mapa físico de la Península Ibérica. Fuente: Emilio Gómez Fernández, Wikimedia Commons, licencia CC BY-SA 3.0.

Esta delimitación es especialmente importante para el presente trabajo, puesto que el problema planteado considera las capitales de las provincias españolas situadas en el territorio peninsular. España está organizada territorialmente en 50 provincias, además de las ciudades autónomas de Ceuta y Melilla. De las 50 provincias, tres son insulares: Illes Balears, Las Palmas y Santa Cruz de Tenerife. Al excluir estas tres provincias se obtienen las 47 provincias peninsulares consideradas en el problema.

Por consiguiente, las ciudades que constituyen los puntos que debe visitar el vendedor son las capitales de esas 47 provincias. El problema no considera Palma, Las Palmas de Gran Canaria ni Santa Cruz de Tenerife debido a su carácter insular. Tampoco se incluyen Ceuta y Melilla, puesto que son ciudades autónomas, no provincias, y además no forman parte de la red de carreteras peninsular.

4.2. Marco teórico

El problema del agente viajero (TSP, por sus siglas en inglés) es uno de los desafíos más estudiados en la optimización combinatoria y las ciencias de la computación. Formalmente, plantea el escenario de un vendedor que debe visitar un conjunto de ciudades determinadas exactamente una vez y, al concluir, retornar a la ciudad de origen. El objetivo matemático principal es identificar la ruta óptima que minimice la distancia total recorrida, el tiempo empleado o los costos financieros del trayecto. A pesar de su enunciado sencillo, el problema posee una enorme complejidad computacional clasificada como NP-hard, lo que significa que el número de soluciones posibles crece de forma factorial a medida que se añaden nuevos destinos, volviendo inviable la evaluación manual o por fuerza bruta para redes de gran escala.(Applegate et al. 2006)

En este caso un vendedor debe visitar las 47 capitales de provincia peninsulares exactamente una vez y regresar a la ciudad de partida. Se busca el orden de visita que haga el recorrido total más corto y luego se tiene en cuenta el costo de desplazamiento entre ciudades.

En este contexto, las colonias de hormigas y los algoritmos genéticos representan dos estrategias diferentes para explorar un espacio de soluciones de gran tamaño. Las primeras utilizan mecanismos de aprendizaje colectivo mediante feromonas, mientras que los segundos emplean una población de soluciones y operadores inspirados en la evolución. La comparación de ambos enfoques permitirá estudiar no solo qué recorrido obtiene el menor costo, sino también cómo evoluciona la búsqueda y qué tan sensibles son los resultados ante cambios en el valor económico del tiempo del vendedor.

Finalmente, el carácter cerrado del recorrido, la incorporación simultánea de tiempo, peajes y combustible, y el análisis del parámetro asociado al valor de una hora de trabajo convierten esta instancia en un problema más cercano a una situación real de planificación de rutas que el TSP puramente geométrico basado únicamente en distancias entre puntos.

4.3. Fundamentación matemática:

Un recorrido es una permutación π de las $n = 47$ capitales, y su longitud incluye el regreso al inicio Applegate et al. 2006:

$$L(\pi) = \sum_{k=1}^{n-1} d_{\pi_k \pi_{k+1}} + d_{\pi_n \pi_1}.$$

Las distancias d_{ij} son kilómetros por carretera calculados con ORS Luxen y Vetter 2011, ejecutado localmente sobre el extracto de OpenStreetMap para España OpenStreetMap contributors 2026; Geofabrik GmbH 2026, de modo que ninguna ruta atraviesa países vecinos. La matriz resultante es *asimétrica* ($d_{ij} \neq d_{ji}$ en general, por sentidos únicos y accesos distintos), así que se trata de un TSP asimétrico.

4.3.1. Colonia de hormigas (ACO)

Cada hormiga construye un recorrido y, estando en la ciudad i, elige la siguiente ciudad j no visitada con probabilidad

$$p_{ij} = \frac{\tau_{ij}^{\alpha}\, \eta_{ij}^{\beta}}{\sum_{l \in \mathcal{N}_i} \tau_{il}^{\alpha}\, \eta_{il}^{\beta}}, \qquad \eta_{ij} = \frac{1}{d_{ij}},$$

donde τ_{ij} es la feromona del arco $i \to j$ (la experiencia acumulada de la colonia) y η_{ij} favorece las ciudades cercanas. Tras cada iteración, la feromona se evapora y se refuerza en los arcos de los recorridos cortos, con un peso extra e para el mejor recorrido global T^{bs}:

$$\tau_{ij} \leftarrow (1-\rho)\, \tau_{ij} + \sum_k \Delta\tau_{ij}^k + e\, \Delta\tau_{ij}^{bs}, \qquad \Delta\tau_{ij}^k = \begin{cases} 1/L_k & \text{si el arco } i \to j \in T^k, \\ 0 & \text{en otro caso.} \end{cases}$$

Como la matriz es asimétrica, solo se refuerza el sentido recorrido (τ_{ij} y no τ_{ji}). La feromona inicial es $\tau_0 = m/L^{nn}$ (Dorigo y Stützle 2004).

4.3.2. Algoritmo genético (GA)

Cada individuo es un recorrido, y su aptitud es $L(\pi)$, que se busca minimizar Holland 1975; Goldberg 1989. En cada generación:

- **Elitismo:** los 2 mejores recorridos pasan intactos.

- **Selección por torneo:** de $k = 5$ recorridos tomados al azar, gana el más corto.
- **Cruce OX** Davis 1985, con $p_c = 0{,}9$: el hijo copia un tramo de un padre y completa el resto en el orden del otro, sin repetir ciudades.
- **Mutación por inversión**, con $p_m = 0{,}3$: se invierte un tramo aleatorio del recorrido.
- **Diversidad:** se descartan los hijos duplicados para evitar la convergencia prematura.

Parámetros: población de 150 y 1 500 generaciones. Como complemento, se evaluó también una variante memética Moscato 1989, que aplica búsqueda local a un 5 % de los hijos y usa 200 generaciones.

4.3.3. Búsqueda local (Para luego hacer comparación)

Ambos algoritmos se combinan con dos movimientos de búsqueda local, que se alternan hasta que ninguno mejora el recorrido. Se aplican a la mejor hormiga de cada iteración y al 5 % de los hijos del GA, lo que convierte a este último en un algoritmo memético (Moscato 1989).

2-opt (Croes 1958). Sustituye dos arcos $a \to b$ y $c \to d$ por $a \to c$ y $b \to d$, lo que obliga a recorrer al revés el tramo intermedio $b, \ldots, c$. Con distancias asimétricas, ese tramo cambia de longitud, así que la variación es

$$\Delta = d_{ac} + d_{bd} - d_{ab} - d_{cd} + \sum_{k=i+1}^{j-1} \left(d_{\pi_{k+1}\pi_k} - d_{\pi_k\pi_{k+1}} \right),$$

donde $\pi_{i+1} = b$ y $\pi_j = c$. El movimiento se acepta si $\Delta < 0$.

Or-opt (Or 1976). Saca un tramo de $\ell \in \{1, 2, 3\}$ ciudades consecutivas $s_1 \to \cdots \to s_\ell$, situado entre p y q, y lo reinserta *sin invertirlo* entre otras dos ciudades consecutivas x e y:

$$\Delta = \left(d_{pq} + d_{xs_1} + d_{s_\ell y} \right) - \left(d_{ps_1} + d_{s_\ell q} + d_{xy} \right).$$

Al conservar el sentido del tramo, es el movimiento natural cuando $d_{ij} \neq d_{ji}$.

4.3.4. Verificación

El óptimo exacto L^* se obtuvo con programación lineal entera en la formulación de Dantzig, Fulkerson y Johnson (1954) Dantzig, Fulkerson y Johnson 1954. Con él se mide la calidad de cada algoritmo:

$$\text{Dif.} = 100 \cdot \frac{L - L^*}{L^*} \,\%$$

5. Provincias españolas y sus capitales

Provincia	Capital
Álava (Araba)	Vitoria-Gasteiz
Albacete	Albacete
Alicante (Alacant)	Alicante (Alacant)
Almería	Almería
Asturias	Oviedo
Ávila	Ávila
Badajoz	Badajoz
Barcelona	Barcelona
Bizkaia (Vizcaya)	Bilbao
Burgos	Burgos
Cáceres	Cáceres
Cádiz	Cádiz
Cantabria	Santander
Castellón (Castelló)	Castellón de la Plana
Ciudad Real	Ciudad Real
Córdoba	Córdoba

Cuadro 7: **De Álava a Córdoba**

Provincia	Capital
A Coruña (La Coruña)	A Coruña
Cuenca	Cuenca
Gipuzkoa (Guipúzcoa)	Donostia / San Sebastián
Girona (Gerona)	Girona
Granada	Granada
Guadalajara	Guadalajara
Huelva	Huelva
Huesca	Huesca
Jaén	Jaén
La Rioja	Logroño
León	León
Lleida (Lérida)	Lleida
Lugo	Lugo
Madrid	Madrid
Málaga	Málaga
Murcia	Murcia

Cuadro 8: **De A Coruña a Murcia**

Provincia	Capital
Navarra	Pamplona (Iruña)
Ourense (Orense)	Ourense
Palencia	Palencia
Pontevedra	Pontevedra
Salamanca	Salamanca
Segovia	Segovia
Sevilla	Sevilla
Soria	Soria
Tarragona	Tarragona
Teruel	Teruel
Toledo	Toledo
Valencia	Valencia
Valladolid	Valladolid
Zamora	Zamora
Zaragoza	Zaragoza

Cuadro 9: **De Navarra a Zaragoza**

Cuadro 10: Provincias y capitales de la España Peninsular

6. Datos geográficos y matrices de tiempo y distancia

6.1. Origen de las coordenadas

El problema se define sobre las 47 capitales de provincia de la España peninsular.

La latitud y la longitud de cada capital se tomaron del *Nomenclátor Geográfico de Municipios y Entidades de Población* (NGMEP) del Centro Nacional de Información Geográfica (Centro Nacional de Información Geográfica (CNIG) s.f.). Es una base de datos oficial del Instituto Geográfico Nacional. Según su página de descarga, las coordenadas de la península están en ETRS89, compatible con WGS84, que es el sistema que usa OpenStreetMap y, por tanto, OpenRouteService. Por eso no fue necesaria ninguna transformación entre sistemas de referencia. La fecha de descarga fue el 4 de octubre del 2026.

Se hicieron tres ajustes antes de usar los datos:

- Se comprobó que hubiera exactamente 47 ciudades, sin nombres repetidos.
- Se guardó la tabla como (latitud, longitud), pero los servicios de enrutamiento esperan el orden (longitud, latitud). Este cambio de orden se hace una sola vez, al construir la solicitud.

6.2. Por qué se necesitan tiempos y distancias por carretera

La distancia en línea recta no representa lo que recorre un vehículo: ignora relieve, trazado y velocidad de cada vía. Como el costo del viaje depende del tiempo y del combustible, hacen falta, para cada par de capitales, el tiempo y la distancia de una ruta real. Con 47 ciudades hay $47 \times 46 = 2162$ pares ordenados. Se calcularon por separado, porque la ida y la vuelta pueden diferir.

6.3. Primer intento: API pública de OpenRouteService

El primer intento usó el servicio *Matrix* (v2, perfil `driving-car`) de la API pública de OpenRouteService (ORS) (GIScience / HeiGIT s.f.[b]). La solicitud enviaba las 47 coordenadas y pedía distancia (km) y duración (s) para todos los pares. Un script en Python hacía la petición, validaba la respuesta (ausencia de valores vacíos, distancia de ajuste a la red, asimetría) y guardaba las matrices D (km) y T (h) junto con un archivo de metadatos.

Cómo calcula ORS cada celda. ORS construye un grafo de carreteras $G = (V, E)$ a partir de OpenStreetMap (GIScience / HeiGIT s.f.[a]; OpenStreetMap Foundation s.f.). Cada coordenada se ajusta (*snapping*) al punto de la red viaria más cercano. A cada tramo e se le asigna una velocidad v_e a partir de las etiquetas de OSM (límite de velocidad, tipo de vía, entorno urbano o rural, superficie) (GIScience / HeiGIT s.f.[d]; GIScience / HeiGIT s.f.[e]), y su tiempo es

$$t_e = \frac{\ell_e}{v_e}.$$

Entre dos puntos se busca la ruta más rápida,

$$P^* = \arg\min_P T(P), \qquad T(P) = \sum_{e \in P} t_e,$$

con un algoritmo de camino mínimo (tipo Dijkstra). La celda T_{ij} es $T(P^*)$ y la celda D_{ij} es la longitud de esa misma ruta. La distancia no se minimiza por separado: es la longitud de la ruta más rápida. No se considera tráfico en tiempo real.

6.4. Error detectado: rutas por carreteras de Portugal

Al revisar las rutas obtenidas se vio que algunos pares no se resolvían por carreteras españolas. En la respuesta de ORS aparecían autopistas portuguesas, en concreto la A 3/IP 1 (*Autoestrada de Entre-Douro-e-Minho*) y la A 2 (*Autoestrada do Sul*).

Por qué ocurre. ORS solo busca el camino más rápido sobre todo el mapa disponible. Si una ruta por Portugal es más rápida que la mejor por España, la elige. Es un resultado correcto para un viajero libre, pero no para este problema, que debe comparar rutas dentro del territorio peninsular español.

Además, la API pública tiene una cuota diaria. En una fase posterior de extracción se recibió el error 403 *Quota exceeded*, lo que mostraba que depender del servicio público limitaba la repetición del cálculo.

6.5. Solución: servidor local de OpenRouteService

Para controlar qué red viaria se usa, se montó una instancia propia de ORS con Docker (GIScience / HeiGIT s.f.[c]):

1. Se descargó de Geofabrik el extracto de OpenStreetMap de España (`spain-latest.osm.pbf`, unos 1,4 GB) (Geofabrik GmbH 2026), el 12 de octubre.
2. Se arrancó ORS en un contenedor con ese archivo como fuente de datos y el perfil `driving-car` activo. Al arrancar, el servicio construye el grafo y lo guarda para reutilizarlo (GIScience / HeiGIT s.f.[c]). Etiqueta de la imagen Docker y versión de ORS.
3. Un script en Python envía una solicitud con el mismo formato que la API pública: las coordenadas en orden (longitud, latitud), `metrics` = distancia y duración, `units` = km y `resolve_locations`. No requiere clave de API.
4. La respuesta se convierte en tablas 47×47 de distancias (km) y de tiempos (s $\div 3600$, es decir, horas), y se guarda junto con las coordenadas y la respuesta cruda.

El motor, el perfil, el modo de calcular tiempos y distancias y el formato de salida son los mismos que en el primer intento. Solo cambia el mapa del que parte el grafo, ahora limitado a España. Para mantener coherencia, todas las matrices usadas en el trabajo se regeneraron con el servidor local; no se mezclan datos de la API pública y del servidor propio.

6.6. Por qué fue importante el cambio

- **Validez del problema.** Con el extracto de España, las rutas ya no pueden usar vías portuguesas; el problema queda restringido a la red viaria española.
- **Distancias y tiempos representativos.** Los 20 pares afectados tenían valores que no corresponden a un trayecto por España, y esos pares se concentran en el oeste y el suroeste, que son los más relevantes para cerrar el ciclo.

6.7. Consecuencias de no haber hecho el cambio

- Las matrices habrían contenido distancias y tiempos de rutas que salen del país, y el costo de esos trayectos se habría calculado sobre carreteras ajenas al problema.
- Los algoritmos (colonia de hormigas y algoritmo genético) optimizan con esos números tal como están. Un atajo artificial por Portugal puede acortar un tramo del recorrido y llevar a un ciclo que parece mejor, pero solo lo es porque la matriz está sesgada.
- El valor del costo total, y el análisis de sensibilidad sobre el valor de la hora del conductor, estarían contaminados por esos pares, y la solución no se podría defender como un recorrido por España.
- Habría quedado una inconsistencia de método: parte de las celdas calculadas con la red española y parte con la red de la península ibérica.

6.8. Supuestos y limitaciones

- Los tiempos salen del modelo de velocidades de ORS, sin tráfico en tiempo real, y pueden diferir de los de la API pública por la configuración y la versión del mapa.
- Los datos de OpenStreetMap son colaborativos y pueden tener errores o etiquetas incompletas (OpenStreetMap Foundation s.f.).

- Las coordenadas corresponden a un punto por capital, y el recorrido real parte de un punto de la red cercano a él.

7. Solución punto 1.

Encuentren el orden óptimo del recorrido usando colonias de hormigas y algoritmos genéticos, todo a partir de la matriz de distancias.

Se ejecutaron las versiones clásicas de ambos algoritmos, sin búsqueda local, sobre la matriz asimétrica de distancias por carretera y con la semilla 42. La colonia de hormigas usó 47 hormigas y 300 iteraciones ($\alpha = 1$, $\beta = 5$, $\rho = 0{,}1$, $e = 5$), y el algoritmo genético, una población de 150 recorridos durante 1 500 generaciones ($p_c = 0{,}9$, $p_m = 0{,}3$, torneo de tamaño $k = 5$, dos individuos de élite). La Tabla 11 resume el resultado.

Cuadro 11: Resumen de resultados (distancias por carretera, semilla 42).

Método	Longitud (km)	Dif. óptimo	Hallada en	Tiempo (s)
Algoritmo genético (GA)	6 257,0	+0,79 %	generación 705	6,9
Colonia de hormigas (ACO)	6 442,5	+3,78 %	iteración 111	2,5
Vecino más cercano	7 360,6	+18,57 %	—	—
Óptimo exacto (PLE)	6 207,7	—	—	—

El mejor recorrido lo encontró el algoritmo genético, con 6 257,0 km. Es un 15,0 % más corto que el de la heurística del vecino más cercano y queda a solo un 0,79 % (49,3 km) del óptimo exacto (6 207,7 km), calculado por programación lineal entera solo como referencia. La colonia de hormigas obtuvo 6 442,5 km, a un 3,78 % del óptimo.

7.1. Mejor recorrido encontrado

Partiendo de Madrid (al ser un ciclo cerrado, el punto de partida no altera la longitud), el mejor recorrido, hallado por el algoritmo genético, es el de la Tabla 12 y la Figura 4.

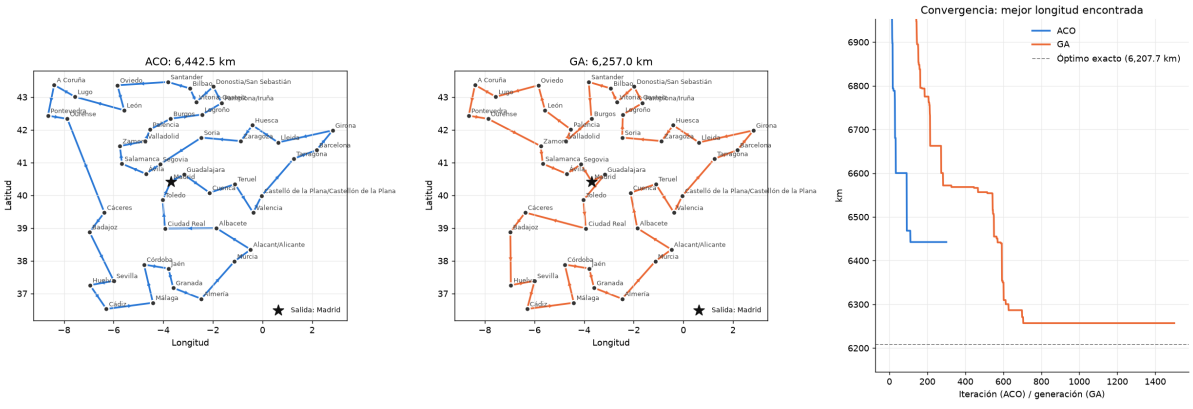


Figura 4: Mejores recorridos hallados por ACO (izquierda) y GA (centro); las flechas indican el sentido. A la derecha, convergencia de la mejor longitud encontrada frente al óptimo exacto (línea discontinua).

Cuadro 12: Mejor recorrido encontrado (GA), tramo a tramo (km por carretera).

#	Tramo	km	Acum.	#	Tramo	km	Acum.
1	Madrid → Guadalajara	57,6	57,6	25	Lleida → Huesca	117,2	3 661,7
2	Guadalajara → Toledo	130,5	188,1	26	Huesca → Zaragoza	73,5	3 735,3
3	Toledo → Ciudad Real	123,0	311,1	27	Zaragoza → Soria	167,8	3 903,1
4	Ciudad Real → Cáceres	299,9	611,1	28	Soria → Logroño	101,3	4 004,5
5	Cáceres → Badajoz	91,2	702,3	29	Logroño → Pamplona	85,8	4 090,2
6	Badajoz → Huelva	234,6	936,8	30	Pamplona → San Sebastián	83,4	4 173,6
7	Huelva → Sevilla	95,3	1 032,2	31	San Sebastián → Vitoria-Gasteiz	101,5	4 275,1
8	Sevilla → Cádiz	121,5	1 153,7	32	Vitoria-Gasteiz → Bilbao	65,4	4 340,6
9	Cádiz → Málaga	234,0	1 387,7	33	Bilbao → Santander	105,2	4 445,8
10	Málaga → Córdoba	159,7	1 547,4	34	Santander → Burgos	180,6	4 626,4
11	Córdoba → Jaén	119,7	1 667,1	35	Burgos → Valladolid	128,0	4 754,4
12	Jaén → Granada	94,3	1 761,4	36	Valladolid → Palencia	49,4	4 803,8
13	Granada → Almería	161,4	1 922,9	37	Palencia → León	133,4	4 937,2
14	Almería → Murcia	222,7	2 145,5	38	León → Oviedo	124,8	5 062,1
15	Murcia → Alicante	81,8	2 227,3	39	Oviedo → Lugo	254,1	5 316,2
16	Alicante → Albacete	168,3	2 395,7	40	Lugo → A Coruña	98,7	5 414,9
17	Albacete → Cuenca	164,3	2 560,0	41	A Coruña → Pontevedra	133,5	5 548,4
18	Cuenca → Teruel	149,1	2 709,1	42	Pontevedra → Ourense	98,5	5 646,9
19	Teruel → Valencia	141,5	2 850,6	43	Ourense → Zamora	259,0	5 905,9
20	Valencia → Castellón	75,5	2 926,1	44	Zamora → Salamanca	67,2	5 973,2
21	Castellón → Tarragona	190,7	3 116,8	45	Salamanca → Ávila	106,8	6 079,9
22	Tarragona → Barcelona	96,2	3 213,0	46	Ávila → Segovia	84,0	6 163,9
23	Barcelona → Girona	103,9	3 316,9	47	Segovia → Madrid	93,2	6 257,0
24	Girona → Lleida	227,6	3 544,5				

Comparación entre algoritmos. Las curvas de convergencia (Figura 4, derecha) muestran dos comportamientos distintos. La colonia de hormigas mejora muy rápido al principio, pero se estabiliza en la iteración 111: la feromona se concentra en un recorrido y la colonia deja de explorar alternativas. El algoritmo genético parte de recorridos aleatorios de unos 25 000 km y mejora de forma más lenta, pero llega más lejos: alcanza su mejor recorrido en la generación 705. Es también más fiable: con diez semillas distintas, el GA obtuvo en promedio 6 292,9 km, entre 6 228,8 y 6 372,5 km.

7.2. Comparación con búsqueda local

Como complemento, se repitieron ambos algoritmos añadiendo una búsqueda local (2-opt y Or-opt) a las mejores soluciones: a la mejor hormiga de cada iteración en ACO y a un 5 % de los hijos en el GA, que pasa así a ser un algoritmo memético. Con esta combinación, ambos alcanzan el óptimo exacto de 6 207,7 km (ACO en la iteración 71 y GA en la generación 4). Esto muestra que las metaheurísticas encuentran la estructura general de un buen recorrido, mientras que la búsqueda local corrige los defectos locales que les impiden llegar al óptimo.

8. Solución punto 2.

9. Parámetros económicos y función de costo

Nota: Durante la investigación se encontró un estudio realizado en 2016 sobre las capitales de provincia de la España peninsular, en el cual se presenta la matriz de distancias entre las diferentes capitales. Aprovechando esta información, se realizará una comparación con los

resultados obtenidos para el año 2026, con el propósito de analizar las posibles diferencias entre ambos escenarios.

El estudio de referencia para el año 2016 se tomó de Roldán 2016.

El costo de cada trayecto entre dos capitales se expresa en euros y combina tres componentes: el tiempo de quien conduce, el combustible del vehículo y los peajes de las autopistas utilizadas. Para convertir las matrices de tiempo y distancia en una matriz de costo es necesario fijar un vehículo de referencia, un precio del combustible, un valor monetario de la hora de trabajo y una matriz de peajes. Esta sección describe cada decisión, la fuente consultada, la fecha de consulta y la razón por la que se adoptó.

9.1. Vehículo de referencia

Se adoptó como vehículo base el **Toyota Corolla Hatchback Hybrid 140H** (Toyota España s.f.). La fecha de consulta de la página fue el 5 de octubre. La elección responde a tres razones:

1. **Es un automóvil convencional.** Se trata de un turismo de uso habitual, de modo que el experimento representa a un vendedor que se desplaza en un coche corriente y no en un vehículo especial.

2. **Dispone de especificaciones oficiales de consumo.** El fabricante publica el consumo combinado bajo el ciclo de homologación WLTP, lo que permite basar el parámetro en un dato verificable y no en una estimación propia.

3. **Usa gasolina.** Funciona con gasolina, de la que existe un precio público y actualizado, y no depende de una infraestructura de recarga cuya disponibilidad variaría entre las capitales del recorrido.

Cuadro 13: Características del vehículo de referencia

Característica	Valor
Marca y modelo	Toyota Corolla Hatchback Hybrid 140H
Sistema de propulsión	Híbrido no enchufable
Motor	1,8 L, ciclo Atkinson, con motor eléctrico
Potencia indicada	140 CV
Combustible	Gasolina 95 E5
Consumo combinado WLTP (rango publicado)	4,4–4,7 L/100 km
Consumo utilizado en el modelo	4,7 L/100 km

Nota. Datos del fabricante (Toyota España s.f.).

Por qué se usa 4,7 L/100 km. El fabricante publica un rango de 4,4 a 4,7 L/100 km. Se tomó el extremo superior, es decir, el consumo más alto del rango oficial. Con ello el costo de combustible no se subestima: los valores WLTP se miden en condiciones de laboratorio y el propio fabricante advierte de que pueden no reflejar el resultado de la conducción real (Toyota España s.f.). Elegir el extremo alto es, por tanto, una decisión prudente y no un promedio.

9.2. Precio del combustible

El precio de la gasolina de 95 octanos se tomó de un agregador que publica los datos oficiales del Ministerio para la Transición Ecológica y el Reto Demográfico (MITECO) (Gasolinera Hoy 2026). Se registró una *captura* (*snapshot*) el **5 de octubre de 2026**, con un valor de **1,799 €/L**, y ese valor se fijó como precio p del modelo.

Se fijó un único precio y no uno distinto por provincia, porque el modelo busca una ruta que sea comparable entre algoritmos y entre experimentos. Un precio por provincia añadiría

una fuente de variación que no es objeto del estudio y haría que el costo de un mismo tramo dependiera del sentido en que se recorre. La fecha de la captura se documenta porque el precio de los carburantes cambia a diario: otro día daría otro valor.

9.3. Conversión de la distancia en costo de combustible

Sea D_{ij} la distancia por carretera, en km, del trayecto de la ciudad i a la ciudad j, obtenida de la matriz de distancias. Con un consumo $q = 4{,}7$ L/100 km y un precio $p = 1{,}799$ €/L se calculan dos magnitudes:

$$L_{ij} = \frac{q\, D_{ij}}{100}, \tag{4}$$

$$F_{ij} = p\, L_{ij}. \tag{5}$$

La ecuación (4) calcula los **litros** que gasta el vehículo en el trayecto. El consumo está expresado por cada 100 km, de modo que se multiplica por la distancia y se divide entre 100: un trayecto de 100 km consume 4,7 L, uno de 200 km consume 9,4 L, y así sucesivamente. La ecuación (5) convierte esos litros en **euros** multiplicándolos por el precio de cada litro.

Al sustituir la primera en la segunda se obtiene un único coeficiente por kilómetro:

$$F_{ij} = p \cdot \frac{q\, D_{ij}}{100} = 1{,}799 \cdot \frac{4{,}7\, D_{ij}}{100} = 0{,}084553\, D_{ij} \quad [€].$$

El coeficiente $k = pq/100 = 0{,}084553$ €/km indica que cada kilómetro recorrido cuesta unos 8,5 céntimos de combustible. Se hace esta simplificación porque el costo de combustible es proporcional a la distancia: no depende de la ciudad concreta, sino solo de cuántos kilómetros se recorren, de modo que basta multiplicar la matriz de distancias por k para obtener la matriz de costo de combustible.

9.4. Peajes

El tercer componente del costo es el peaje P_{ij}, expresado en euros, asociado al desplazamiento de la ciudad i a la ciudad j. Se incorpora a la función de costo (ecuación (9)) para que el modelo valore, además del tiempo y del combustible, el dinero que se paga por el uso de autopistas.

Dificultades en la obtención de los datos. A diferencia de las distancias y los tiempos de conducción, obtenidos con el servidor local de OpenRouteService, no se contaba con una matriz completa y verificable de tarifas de peaje para los 47 destinos. Además, el importe de un peaje depende de la autopista utilizada, del tramo recorrido, de la categoría del vehículo y, en determinadas vías, de las condiciones tarifarias aplicables (Secretaría General de Transporte Terrestre 2025). Por ello no era posible asignar una tarifa única y exacta a cada uno de los $47 \times 46 = 2162$ pares de capitales sin información detallada sobre las rutas y sus tramos de peaje.

Solución adoptada: una matriz sintética. Para no dejar desplazamientos sin valorar y mantener un procedimiento de cálculo uniforme, se construyó una matriz de peajes **sintética**, que aproxima el peaje de cada desplazamiento a partir de la distancia por carretera entre las dos capitales. Esta matriz no reproduce necesariamente el importe real que se pagaría en una ruta concreta y, por esa razón, el archivo que la contiene se identifica expresamente como sintético.

Supuestos.

1. **Proporción de trayecto con peaje.** Se supuso que el 20 % de la distancia de cada trayecto corresponde a vías de peaje. Este valor es un supuesto de modelización y no una estadística oficial; no se apoya en mediciones de los recorridos individuales.

2. **Tarifa de referencia de 2025.** Se tomó la tarifa oficial para vehículos ligeros sin dispositivo de telepeaje (sin TAG) en las autopistas del Estado, publicada en el BOE (Secretaría General de Transporte Terrestre 2024): 0,0927 €/km, sin IVA. Se eligió la tarifa sin TAG porque el vehículo de referencia se considera sin dicho dispositivo; además, es la más alta para vehículos ligeros (con TAG es de 0,0827 €/km), por lo que no subestima el peaje.

3. **Actualización a 2026.** La misma resolución fija un incremento máximo anual del 2 % para 2026 (Secretaría General de Transporte Terrestre 2024), de modo que la tarifa de referencia de 2026 es
$$t_{2026} = 0{,}0927 \times 1{,}02 = 0{,}094554 \text{ €/km}. \tag{6}$$

Cálculo. Como solo una parte del trayecto se supone sujeta a peaje, la tarifa se reparte sobre la distancia total para obtener una tarifa efectiva media por kilómetro recorrido:
$$r_{\text{efectiva}} = 0{,}20 \times t_{2026} = 0{,}20 \times 0{,}094554 = 0{,}0189108 \text{ €/km}. \tag{7}$$
Esta ecuación indica que, en promedio, cada kilómetro del trayecto aporta 0,0189 € de peaje, aunque la mayor parte de los kilómetros no tenga peaje. El peaje sintético entre las ciudades i y j es entonces
$$P_{ij} = D_{ij} \times r_{\text{efectiva}} = 0{,}0189108\, D_{ij}, \qquad P_{ii} = 0, \tag{8}$$
donde D_{ij} es la distancia por carretera, en kilómetros, obtenida del servidor local de OpenRouteService.

Interpretación y limitaciones.

- La proporción del 20 % y la tarifa de referencia son supuestos de modelización, no mediciones de cada recorrido. Los valores obtenidos son, por tanto, estimaciones sintéticas y no tarifas reales verificadas para cada trayecto.
- La tarifa de referencia corresponde a un grupo de autopistas del Estado y no representa todas las autopistas españolas, cuyas tarifas difieren según el tramo y el vehículo (Secretaría General de Transporte Terrestre 2024; Secretaría General de Transporte Terrestre 2025).
- Como P_{ij} es proporcional a la distancia, el peaje equivale a sumar 0,0189 € por kilómetro al costo de combustible, que es de 0,0846 €/km. La función de costo queda $C_{ij} = V_h T_{ij} + 0{,}103464\, D_{ij}$. Esto significa que el modelo no distingue entre una ruta con peaje y otra libre: el peaje encarece de forma proporcional los trayectos largos, pero no recoge el caso en que una autopista de pago ahorra tiempo a cambio de dinero.
- La ventaja del procedimiento es que es homogéneo, reproducible y deja todos los pares de capitales valorados.

9.5. Valor de la hora

Qué se mide y de dónde se tomó. El tiempo de viaje no es gratuito: durante las horas de conducción el vendedor no atiende clientes ni realiza otras tareas. Para asignarle un valor se usó el **coste laboral por hora efectiva** publicado por el Instituto Nacional de Estadística (INE) en la Encuesta Trimestral de Coste Laboral del segundo trimestre de 2026, que fue de **26,18 €/h** (Instituto Nacional de Estadística 2026). El INE publicó este dato el 17 de septiembre de 2026.

Por qué coste laboral y no salario. Se distingue entre *salario* y *coste laboral*. El salario es lo que recibe el trabajador. El coste laboral es lo que cuesta ese trabajador a la empresa, e incluye, además del salario, otros costes como las cotizaciones obligatorias a la Seguridad Social. En la misma publicación el INE desglosa un coste laboral total de 3 388,15 € por trabajador y mes en un coste salarial de 2 512,68 € y otros costes de 875,47 € (la suma coincide con el total) (Instituto Nacional de Estadística 2026). Como el costo que se modela es el de la empresa que paga las horas de viaje, se usó el coste laboral, que es mayor que el salario. Además, el INE lo expresa por *hora efectiva*, es decir, por hora realmente trabajada, y no por hora pagada, lo que evita subestimar el valor de una hora productiva.

Cómo debe interpretarse. El valor de 26,18 €/h es un promedio nacional del coste laboral por hora y no el de una profesión concreta. Por eso se utiliza como **parámetro de modelación**: un valor base razonable y verificable, y no una constante válida para cualquier vendedor. Un vendedor con mayor o menor remuneración tendría otro valor, y precisamente por ello el estudio incluye un análisis de sensibilidad sobre este parámetro (Sección **??**).

9.6. Función de costo completa

El costo de recorrer el trayecto de i a j suma los componentes anteriores:

$$C_{ij} = V_h\, T_{ij} + P_{ij} + k\, D_{ij}, \qquad V_h = 26{,}18, \quad k = 0{,}084553, \tag{9}$$

donde T_{ij} es el tiempo de viaje en horas, D_{ij} la distancia en km, y P_{ij} y C_{ij} están en euros. Los tres sumandos representan, en orden, el costo del tiempo del conductor, el de los peajes y el del combustible. La matriz P se describe en la Sección 9.4.

9.7. Sensibilidad al valor de la hora

El objetivo del análisis es identificar a partir de qué valor económico de la hora del vendedor cambia la ruta óptima, es decir, cuánto debe valer una hora de trabajo para que el algoritmo seleccione un recorrido diferente. Sin embargo, en el modelo el tiempo de desplazamiento se calcula a partir de la distancia entre ciudades y de una velocidad media constante de 80 km/h. Este cálculo permite estimar el tiempo total de cada ruta, pero no contempla variaciones de la velocidad según la hora del día. Por tanto, el modelo no permite determinar a qué hora de salida cambia la ruta óptima.

9.8. Limitaciones

- El precio de la gasolina corresponde a un día concreto (5 de octubre de 2026) y varía a diario; se fijó un valor único para todo el territorio.
- El consumo de 4,7 L/100 km es un dato de homologación y no un consumo medido en carretera; el real depende de la velocidad, la carga y el relieve.
- El valor de 26,18 €/h es una media nacional del coste laboral y no el de un perfil de vendedor concreto.
- El modelo supone un vehículo y una tarifa de tiempo constantes en todos los trayectos y no incluye otros costes como el desgaste, el seguro o las dietas.

Cuadro 14: Mejor recorrido encontrado por cada algoritmo y escenario (semilla 20261008).

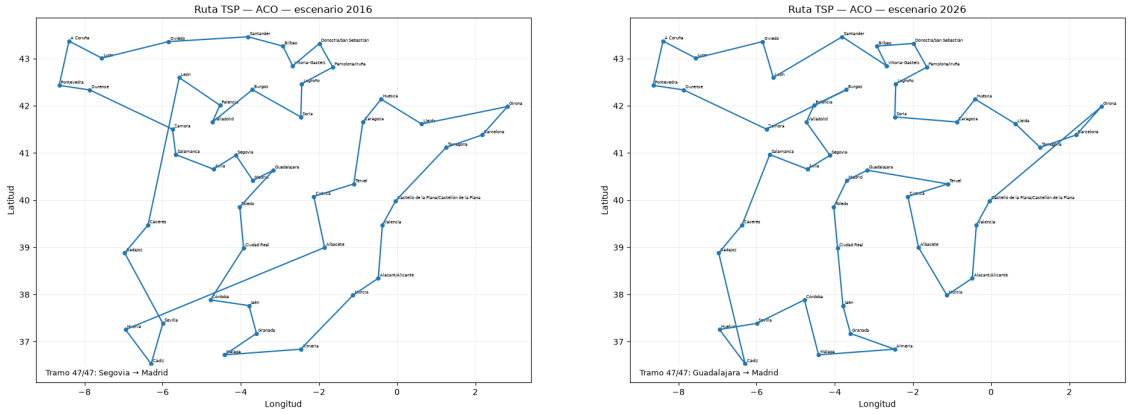
Escenario	Algoritmo	Costo (€)	km	Horas	Tiempo (€)	Combustible (€)	Peajes (€)
2016	ACO	**2169,93**	6836,0	85,5	1689,35	375,91	104,67
2016	GA	2515,84	7929,0	99,1	1959,45	436,02	120,37
2026	ACO	**2905,02**	6742,7	84,3	2206,54	570,11	128,37
2026	GA	3574,53	8295,3	103,7	2714,63	701,39	158,50

9.9. Resultados

En ambos escenarios, la colonia de hormigas (ACO) obtuvo el recorrido de menor costo: 2169,93 € en 2016 y 2905,02 € en 2026. El GA quedó un 15,9 % y un 23,0 % por encima, respectivamente (345,92 € y 669,51 € más). El recorrido del ACO es además más corto (1093 y 1553 km menos) y más rápido (13,7 y 19,4 horas menos).

La diferencia se explica por cómo construye cada algoritmo sus soluciones. El ACO arma cada recorrido ciudad a ciudad, prefiriendo los tramos de menor costo ($\eta_{ij} = 1/c_{ij}$), así que incluso sus primeras soluciones son razonables. El GA parte de permutaciones aleatorias y solo las mejora recombinándolas. Con una población de 60 individuos y 150 generaciones no llega a eliminar los tramos largos. Esto se ve en sus recorridos: Alicante → Girona (615 km) y Córdoba → Salamanca (529 km) en 2016, o Tarragona → Guadalajara (486 km) en 2026, son saltos que cruzan la península y que una ruta eficiente evitaría.

El costo del mejor recorrido aumenta un 33,9 % entre 2016 y 2026 (de 2169,93 € a 2905,02 €), aunque la distancia recorrida es similar (6836 frente a 6743 km). El aumento se debe a los precios. El componente de tiempo sube un 30,6 %, por el valor de la hora (de 19,77 a 26,18 €/h). El combustible sube un 51,7 %, por el precio (de 1,17 a 1,799 €/L). Los peajes suben un 22,6 %.



(a) 2016: 2169,93 € (6836 km, 85,5 h)

(b) 2026: 2905,02 € (6742,7 km, 84,3 h)

Figura 5: Mejor recorrido encontrado en cada escenario, obtenido en ambos casos con ACO (semilla 20261008).

La Figura 5 muestra las mejores rutas de cada escenario, ambas obtenidas con ACO. Son las mejores porque, en los dos años, tienen el menor costo total frente al GA: 2169,93 € frente a 2515,84 € en 2016 (Figura 5a) y 2905,02 € frente a 3574,53 € en 2026 (Figura 5b). Además, recorren menos kilómetros y tardan menos horas. Visualmente, ambas rutas forman un circuito que bordea la península mientras que las del GA atraviesan el país con tramos de más de 480 km. Aunque las dos rutas recorren España en sentidos distintos, su distancia es casi igual

(6836 y 6742,7 km). El aumento del costo en 2026 se debe, por tanto, a los precios del tiempo y del combustible, no a la ruta.

10. USO DE IA

10.1. Punto 1

A continuación se presentan los principales *prompts* utilizados durante el desarrollo del trabajo. Estos permitieron apoyar la comprensión matemática de las funciones de prueba, la implementación computacional de los algoritmos de optimización, el diseño de la arquitectura del código y la definición del protocolo experimental empleado para comparar los diferentes métodos.

Prompt 1: Análisis Teórico y Derivación del Gradiente

"Actúa como un experto en optimización matemática. Necesito comprender y documentar las funciones de Rosenbrock y Rastrigin.

1. Explica la geometría, el dominio habitual, el mínimo global y la formulación matemática de cada función en 2D, 3D y d dimensiones.
2. Realiza la derivación paso a paso de cada función en 2D, aplicando explícitamente la regla de la cadena para obtener las derivadas parciales respecto a x_1 y x_2.
3. Muestra el vector gradiente final $\nabla f(x_1, x_2)$ de cada función y evalúalo numéricamente en su punto óptimo, verificando que sea igual al vector nulo. Considera $(1, 1)$ para Rosenbrock y $(0, 0)$ para Rastrigin.

"

Prompt 2: Implementación de las funciones objetivo "Escribe en Python una implementación vectorizada de las funciones de Rosenbrock y Rastrigin.

La implementación debe cumplir con los siguientes requisitos:

- Utilizar la librería `numpy` de forma vectorizada.
- Funcionar para cualquier dimensión $d \geq 2$.
- Validar que la dimensión del vector de entrada sea consistente con el parámetro de dimensión especificado.
- Mantener una estructura modular que permita utilizar ambas funciones como problemas de prueba dentro del mismo framework de optimización.

"

Prompt 3: Arquitectura Modular de Optimización "Tengo implementadas las funciones objetivo de Rosenbrock y Rastrigin y no debo modificar sus definiciones.

Diseña una arquitectura modular en Python que cumpla con:

1. Cada algoritmo de optimización (Descenso por Gradiente, PSO, Evolución Diferencial, etc.) debe implementarse como una función independiente.
2. Debe existir una función principal
   `optimize(algorithm, problem, dim, bounds, random_seed, max_evals)`
   que permita seleccionar el algoritmo y la función objetivo.
3. Debe soportar problemas en dos y tres dimensiones.
4. Garantizar la reproducibilidad mediante el control centralizado de semillas aleatorias.
5. Registrar automáticamente el número de evaluaciones de la función objetivo (n_f) y del gradiente (n_{grad}).

"

Prompt 4: Diseño Experimental "Diseña un protocolo experimental para comparar los algoritmos implementados sobre las funciones de Rosenbrock y Rastrigin en dos y tres dimensiones.

El protocolo debe:

- Ejecutar al menos 30 corridas independientes por algoritmo utilizando diferentes semillas aleatorias.
- Implementar la métrica de evaluaciones equivalentes

$$n_{\text{equiv}} = n_f + 2d\, n_{\text{grad}},$$

  asumiendo diferencias finitas centrales.
- Reportar una tabla con:
  - Media
  - Desviación estándar
  - Mejor valor encontrado
  - Peor valor encontrado
  - Tasa de éxito utilizando un umbral de convergencia previamente definido.
- Sugerir una forma adecuada de comparar el desempeño de algoritmos basados en gradiente y algoritmos metaheurísticos.

"

El uso de herramientas de Inteligencia Artificial generativa apoyó diferentes etapas del desarrollo del proyecto, desde la comprensión de los conceptos teóricos hasta la implementación computacional y el diseño de los experimentos. Sin embargo, todas las respuestas obtenidas fueron revisadas y validadas antes de ser incorporadas al trabajo final.

A. Comprensión del Problema y Diseño de la Solución

- **Apoyo conceptual:** La IA facilitó la comprensión de las propiedades matemáticas de las funciones de prueba y de los algoritmos de optimización, proporcionando explicaciones sobre su geometría, comportamiento y características principales.
- **Diseño modular:** También sirvió como apoyo para proponer una arquitectura de código organizada y modular, favoreciendo la separación entre las funciones objetivo, los algoritmos de optimización y el protocolo experimental, lo que facilitó el mantenimiento y la extensión del proyecto.

B. Implementación y Verificación

- **Apoyo en la programación:** La IA ayudó a resolver dudas relacionadas con la implementación de funciones, el uso de bibliotecas como `numpy` y la organización del código siguiendo buenas prácticas de programación.
- **Verificación matemática y computacional:** Asimismo, permitió revisar derivaciones, expresiones matemáticas y resultados intermedios, contribuyendo a detectar posibles errores antes de realizar las pruebas experimentales.

C. Diseño Experimental y Análisis de Resultados

- **Planeación de experimentos:** La IA apoyó la definición del protocolo experimental, la organización de las corridas independientes y la selección de métricas para comparar el desempeño de los algoritmos.
- **Interpretación de resultados:** También contribuyó a la estructuración de tablas, figuras y criterios para analizar el comportamiento de los diferentes métodos de optimización, facilitando la elaboración del reporte técnico.

10.2. Punto 2

A continuación, se clasifican los *prompts* más relevantes utilizados a lo largo de las distintas fases:

10.3. Fase 1: Comprensión del Problema y Contexto Teórico

- **Investigación y Enfoque Inicial:** *"Resulta que tengo el siguiente trabajo en mi materia de redes neuronales y algoritmos bioinspirados, en primer lugar no quiero que me digas la solución, si no que me ayudes a entender el problema y los sitios donde debería investigar para llegar a un buen desarrollo, tanto teórico como práctico."*
- **Contexto Geográfico:** *"Necesito aplicar este problema a la España peninsular. ¿Cuáles son las consideraciones geográficas que debo tener en cuenta al buscar las coordenadas de las 47 capitales para que el modelo funcione por carretera?"*

10.4. Fase 2: Obtención de Datos y Detección de Errores

- **Consulta de API Pública:** *"Tengo un Excel con la latitud y longitud de las 47 capitales. ¿Cómo puedo usar Python y la API de OpenRouteService para iterar sobre estas coordenadas y obtener una matriz cuadrada de distancias y tiempos?"*
- **Detección del Problema Transfronterizo:** *"Al revisar las distancias generadas por la API entre ciudades como Huelva y Pontevedra, noto que los tiempos son muy bajos porque la ruta cruza por Portugal. ¿Cómo puedo restringir la API para que solo calcule rutas dentro de España?"*

10.5. Fase 3: Implementación del Servidor Local (Docker)

- **Diseño de la Solución Local:** *"Para evitar que las rutas pasen por Portugal, me recomendaron usar un archivo '.osm.pbf' solo de España y montar un servidor local. Explícame paso a paso cómo usar Docker para configurar OpenRouteService en mi computadora con este mapa."*
- **Resolución de Errores de Entorno:** *"Al intentar correr el contenedor de Docker en Windows, me sale el error 'Output files can not be written' y el servidor se apaga. ¿Cómo soluciono este problema de permisos usando volúmenes de Docker?"*
- **Script Cliente-Servidor:** *"El servidor de Docker ya está corriendo en localhost:8080. Escribe el script de Python que envíe las coordenadas de mi Excel a esta API local y exporte las matrices de tiempo y distancia en archivos CSV."*

11. Resultados, discusión y conclusiones

11.1. Resultados del proceso

El proceso permitió construir una base de datos propia y documentada sobre las 47 capitales de provincia de la España peninsular: coordenadas, matrices de distancia y de tiempo por carretera, parámetros económicos (vehículo, combustible y valor de la hora) y una matriz de peajes. Cada dato tiene una fuente y una fecha de consulta.

11.2. Discusión

El caso de Portugal mostró que una herramienta puede devolver resultados aparentemente correctos que no sirven para el problema planteado. OpenRouteService busca la ruta más rápida sobre todo el mapa disponible y, si una ruta por Portugal era más rápida, la elegía. En un problema de optimización, un solo atajo falso puede cambiar qué ruta parece mejor. El servidor local resolvió el error.

La parte de los peajes mostró que no siempre existe el dato necesario: el peaje depende de la autopista, del tramo y del vehículo, y no se disponía de información de las rutas. Se optó por una aproximación declarada como tal. Al ser proporcional a la distancia, esta aproximación no distingue una ruta con peaje de una libre, por lo que sus valores deben leerse como estimaciones y no como tarifas reales.

11.3. Conclusiones

La calidad de los resultados depende de la calidad de los datos. Antes de aplicar cualquier algoritmo fue necesario verificar el origen de las coordenadas, la red de carreteras utilizada y el significado de cada parámetro; el error de las rutas por Portugal se detectó al revisar los resultados con cuidado y no era evidente en las matrices finales.

Documentar cada decisión, con su fuente, su fecha, su razón y sus límites, forma parte del trabajo: permite que otras personas lo entiendan, lo repitan y juzguen hasta dónde llegan los resultados. Cuando un dato no pudo obtenerse, la solución fue construir una aproximación y declararla abiertamente. Por último, muchos parámetros del modelo son decisiones y no valores fijos, por lo que conviene estudiar cómo responde el modelo cuando cambian.

Referencias

Applegate, David L. et al. (2006). *The Traveling Salesman Problem: A Computational Study*. Princeton, New Jersey: Princeton University Press.

Centro Nacional de Información Geográfica (CNIG) (s.f.). *Nomenclátor Geográfico de Municipios y Entidades de Población*. Instituto Geográfico Nacional. URL: https://centrodedescargas.cnig.es/CentroDescargas/nomenclator-geografico-municipios-entidades-poblacion.

Croes, G. A. (1958). «A Method for Solving Traveling-Salesman Problems». En: *Operations Research* 6.6, págs. 791-812. DOI: 10.1287/opre.6.6.791.

Dantzig, George, Ray Fulkerson y Selmer Johnson (1954). «Solution of a Large-Scale Traveling-Salesman Problem». En: *Journal of the Operations Research Society of America* 2.4, págs. 393-410. DOI: 10.1287/opre.2.4.393.

Davis, Lawrence (1985). «Applying Adaptive Algorithms to Epistatic Domains». En: *Proceedings of the 9th International Joint Conference on Artificial Intelligence (IJCAI-85)*. Vol. 1. Los Angeles, California: Morgan Kaufmann, págs. 162-164.

Dorigo, Marco y Thomas Stützle (2004). *Ant Colony Optimization*. Cambridge, Massachusetts: MIT Press.

Gasolinera Hoy (5 de oct. de 2026). *Precio de la gasolina 95 en España*. Datos atribuidos a MITECO. URL: https://gasolinerahoy.es/.

Geofabrik GmbH (2026). *OpenStreetMap Data Extracts: Spain*. URL: https://download.geofabrik.de/europe/spain.html.

GIScience / HeiGIT (s.f.[a]). *Data: Input and Output Folders and Files*. openrouteservice Backend Documentation. URL: https://giscience.github.io/openrouteservice/run-instance/data.

— (s.f.[b]). *Matrix Endpoint*. openrouteservice Backend Documentation. URL: https://giscience.github.io/openrouteservice/api-reference/endpoints/matrix/.

— (s.f.[c]). *Running with Docker*. openrouteservice Backend Documentation. URL: https://giscience.github.io/openrouteservice/run-instance/running-with-docker.

— (s.f.[d]). *Tag Filtering*. openrouteservice Backend Documentation. URL: https://giscience.github.io/openrouteservice/technical-details/tag-filtering.

— (s.f.[e]). *Travel Speeds*. openrouteservice Backend Documentation. URL: https://giscience.github.io/openrouteservice/technical-details/travel-speeds/.

Goldberg, David E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, Massachusetts: Addison-Wesley.

Holland, John H. (1975). *Adaptation in Natural and Artificial Systems*. Ann Arbor, Michigan: University of Michigan Press.

Instituto Nacional de Estadística (17 de sep. de 2026). *Encuesta Trimestral de Coste Laboral: segundo trimestre de 2026*. URL: https://www.ine.es/dyngs/Prensa/ETCL2T26.htm.

Luxen, Dennis y Christian Vetter (2011). «Real-Time Routing with OpenStreetMap Data». En: *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*. Chicago, Illinois: ACM, págs. 513-516. DOI: 10.1145/2093973.2094062.

Moscato, Pablo (1989). *On Evolution, Search, Optimization, Genetic Algorithms and Martial Arts: Towards Memetic Algorithms*. C3P Report 826. Pasadena, California: California Institute of Technology.

OpenStreetMap contributors (2026). *OpenStreetMap*. URL: https://www.openstreetmap.org.

OpenStreetMap Foundation (s.f.). *Copyright and License*. URL: https://www.openstreetmap.org/copyright.

Or, Ilhan (1976). «Traveling Salesman-Type Combinatorial Problems and Their Relation to the Logistics of Regional Blood Banking». Tesis doct. Evanston, Illinois: Northwestern University.

Rastrigin, L. A. (1974). *Sistemy ekstremal'nogo upravleniya*. Moscow: Nauka.

Roldán, Pedro López (2016). *metinvsoccua$_c$ap3 − 11a2016v3*. Scribd. Consultado el 13 de octubre de 2026. URL: https://es.scribd.com/document/368696521/metinvsoccua-cap3-11a2016v3.

Rosenbrock, H. H. (1960). «An automatic method for finding the greatest or least value of a function». En: *The Computer Journal* 3.3, págs. 175-184. DOI: 10.1093/comjnl/3.3.175.

Secretaría General de Transporte Terrestre (6 de dic. de 2024). *Resolución de 6 de diciembre de 2024, de la Secretaría General de Transporte Terrestre, por la que se publica el Acuerdo del Consejo de Ministros de 3 de diciembre de 2024, por el que se aprueban las tarifas aplicables en determinadas autopistas de la Red de Carreteras del Estado*. Boletín Oficial del Estado. URL: https://www.boe.es/diario_boe/txt.php?id=BOE-A-2024-26469.

— (2025). *Informe 2024 sobre las autopistas de peaje y el sector concesional de carreteras en España*. Informe técnico. Centro de Publicaciones. URL: https://publicaciones.transportes.gob.es/informe-2024-sobre-las-autopistas-de-peaje-y-el-sector-concesional-de-carreteras-en-espana-delegacion-del-gobierno-en-las-sociedades-concesionarias-de-autopistas-nacionales-de-peaje.

Toyota España (s.f.). *Toyota Corolla*. URL: https://www.toyota.es/coches/corolla.